

## ADVANCE PROBLEM PROGRAMMING

### SLO 2: Multi-Paradigm Use

#### 1. Write a Java Program Using Both Class and Lambda (OOP + Functional)

```
import java.util.function.Function;
```

```
class Calculator {
```

```
    int number;
```

```
    Calculator(int number) {
```

```
        this.number = number;
```

```
    }
```

```
    void display() {
```

```
        System.out.println("Number = " + number);
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Calculator c = new Calculator(5);
```

```
        c.display();
```

```
        Function<Integer, Integer> square = n -> n * n;
```

```
        System.out.println("Square = " + square.apply(c.number));
```

```
    }
```

```
}
```

**Output:**

Number = 5

Square = 25

Here, the Calculator class and object demonstrate **Object-Oriented Programming**, while the lambda expression demonstrates **Functional Programming**.

---

## 2. Why is Java Considered a Multi-Paradigm Language?

Java is considered a **multi-paradigm language** because it supports multiple programming styles within the same language.

Java supports:

1. **Object-Oriented Programming** – through classes, objects, inheritance, polymorphism, and encapsulation.
2. **Procedural/Imperative Programming** – through variables, loops, conditional statements, and methods.
3. **Functional Programming** – through lambda expressions, functional interfaces, and the Stream API.
4. **Generic Programming** – through generics such as `ArrayList<Integer>` and generic methods.

Therefore, Java allows programmers to combine different paradigms depending on the requirements of the problem.

---

## 3. Convert a Procedural Java Code into an Object-Oriented Version

**Procedural Version:**

```
public class Main {
```

```
    static int calculateArea(int length, int width) {
```

```
        return length * width;
    }

    public static void main(String[] args) {
        int length = 10;
        int width = 5;

        System.out.println("Area = " + calculateArea(length, width));
    }
}
```

**Output:**

Area = 50

**Object-Oriented Version:**

```
class Rectangle {
    private int length;
    private int width;

    Rectangle(int length, int width) {
        this.length = length;
        this.width = width;
    }

    int calculateArea() {
        return length * width;
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Rectangle r = new Rectangle(10, 5);  
  
        System.out.println("Area = " + r.calculateArea());  
    }  
}
```

**Output:**

Area = 50